

UVa 10344 - 23 Out of 5

1. Explicación del Problema

El objetivo de este problema consiste en determinar si es posible estructurar una expresión aritmética utilizando exactamente cinco números enteros positivos proporcionados, ordenados en cualquier secuencia posible, de modo que el resultado final de la evaluación sea idéntico a **23**.

Los operadores aritméticos permitidos son únicamente la suma (+), la resta (-) y la multiplicación (×). Un factor crucial del problema es que el orden de evaluación viene impuesto estrictamente por una jerarquía asociativa de izquierda a derecha definida por los paréntesis de la fórmula:

$$(((a_{\pi(1)} \circ_1 a_{\pi(2)}) \circ_2 a_{\pi(3)}) \circ_3 a_{\pi(4)}) \circ_4 a_{\pi(5)}$$

Esto significa que se descartan por completo las reglas estándar de precedencia de operadores (donde la multiplicación normalmente se procesaría antes que una suma). En su lugar, las operaciones se calculan de manera secuencial pura a medida que se recorren los números. Dado que el total de números es siempre **5** y el conjunto de operadores es pequeño, la solución óptima y directa involucra dos fases recursivas consecutivas:

- Generar de forma exhaustiva todas las permutaciones posibles de los cinco números de entrada.
- Para cada permutación individual obtenida, intercalar recursivamente los tres operadores disponibles en los cuatro espacios intermedios, calculando el valor acumulado en tiempo real.

2. Código Fuente en C++

```
#include <iostream>
using namespace std;

int numeros[5];
int permutacion_actual[5];
bool usado[5];
bool alcanzado;

// Función recursiva para evaluar los 3 operadores (+, -, *) de izquierda a derecha
void evaluarOperadores(int indice, int valor_acumulado) {
    // Si ya logramos el 23 en otra rama, no seguimos buscando
    if (alcanzado) return;

    // Si ya procesamos los 5 números (4 operaciones completadas)
    if (indice == 5) {
        if (valor_acumulado == 23) {
            alcanzado = true;
        }
        return;
    }

    int siguiente_numero = permutacion_actual[indice];

    // Intentar con Suma
```

```

    evaluarOperadores(indice + 1, valor_acumulado + siguiente_numero);

    // Intentar con Resta
    evaluarOperadores(indice + 1, valor_acumulado - siguiente_numero);

    // Intentar con Multiplicación
    evaluarOperadores(indice + 1, valor_acumulado * siguiente_numero);
}

// Función recursiva para generar manualmente todas las permutaciones de los 5 números
void generarPermutaciones(int posicion) {
    if (alcanzado) return;

    // Si ya colocamos los 5 números en un orden específico, evaluamos los operadores
    if (posicion == 5) {
        // Iniciamos la evaluación con el primer número como valor acumulado inicial
        evaluarOperadores(1, permutacion_actual[0]);
        return;
    }

    // Probar colocar cada uno de los 5 números de entrada en la posición actual
    for (int i = 0; i < 5; i++) {
        if (!usado[i]) {
            usado[i] = true;
            permutacion_actual[posicion] = numeros[i];

            generarPermutaciones(posicion + 1);

            // Backtrack para liberar el número y probar otra combinación
            usado[i] = false;
        }
    }
}

int main() {
    while (true) {
        int ceros = 0;
        for (int i = 0; i < 5; i++) {
            cin >> numeros[i];
            if (numeros[i] == 0) {
                ceros++;
            }
        }

        // Si los cinco números son 0, terminamos la ejecución del programa
        if (ceros == 5) {
            break;
        }

        alcanzado = false;
        for (int i = 0; i < 5; i++) {
            usado[i] = false;
        }

        // Ejecutar algoritmo de búsqueda exhaustiva
        generarPermutaciones(0);

        if (alcanzado) {

```

```

        cout << "Possible" << endl;
    } else {
        cout << "Impossible" << endl;
    }
}

return 0;
}

```

3. Explicación del Código

El algoritmo mantiene un enfoque elemental de programación estructurada, evitando dependencias externas de librerías STL:

- **Lectura y Control de Parada:** El flujo de entrada procesa tuplas de 5 enteros dentro de un ciclo infinito. Se utiliza una variable contadora para verificar si todos los elementos ingresados equivalen a cero; en cuyo caso, el programa efectúa un corte de ejecución inmediato (`break`), ignorando dicha línea de conformidad con las especificaciones del problema.
- **Permutaciones Manuales:** En lugar de delegar el reordenamiento a funciones prefabricadas, la función `generarPermutaciones(int posicion)` construye los arreglos de ordenamiento de forma manual. Para ello, se asiste de un arreglo de banderas booleanas `usado[5]`, el cual evita seleccionar un mismo operando más de una vez en el mismo ciclo.
- **Evaluación Sucesiva de Operadores:** Cuando una permutación se completa (`posicion == 5`), se invoca a `evaluarOperadores(int indice, int valor_acumulado)`. Esta función recibe el estado actual del cálculo y ramifica en tres llamadas recursivas simultáneas, correspondientes a probar de manera exhaustiva la adición, sustracción y multiplicación del número localizado en el índice en evaluación.
- **Optimización por Flag Global:** Se define la variable booleana global `alcanzado`. En el instante preciso en que cualquier combinación aritmética resulta en un valor de **23**, esta bandera se vuelve verdadera. Gracias a las condiciones de control al inicio de ambas funciones de búsqueda, la presencia de este flag provoca un colapso inmediato y ordenado de toda la pila de recursión pendiente, asegurando eficiencia y deteniendo cálculos innecesarios.